

实验四 —— 防火墙实验

班级：2022211801 学号：2022211570 姓名：项枫

一、配置 linux 系统下 iptables 防火墙实验

（一）实验内容

1、学习 iptables 防火墙基本操作。

2、设置 iptables 防火墙的包过滤规则，分别实现以下功能：禁止所有主机 ping 本地主机；仅允许某特定 IP 主机 ping 本地主机；允许每 10 秒钟通过 1 个 ping 包；阻断来自某个 mac 地址的数据包。

3、设置 iptables 规则，实现特定远端主机 SSH 连接本地主机。

（二）实验环境

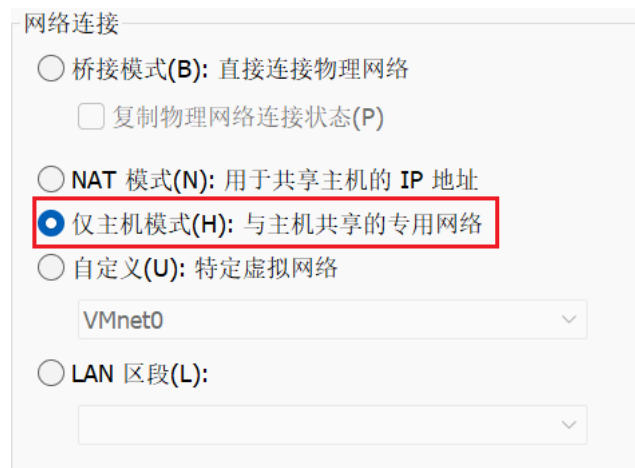
三台虚拟机：1) Kali (192.168.44.129) 配置防火墙；2) WinXP1 (192.168.44.131)、WinXP2 (192.168.44.132) 验证防火墙功能

防火墙：linux 内核防火墙的命令行工具 iptables

Windows 平台 SSH 工具：Xshell5

（三）实验步骤

1、配置实验虚拟机网络连接均设置为仅主机模式

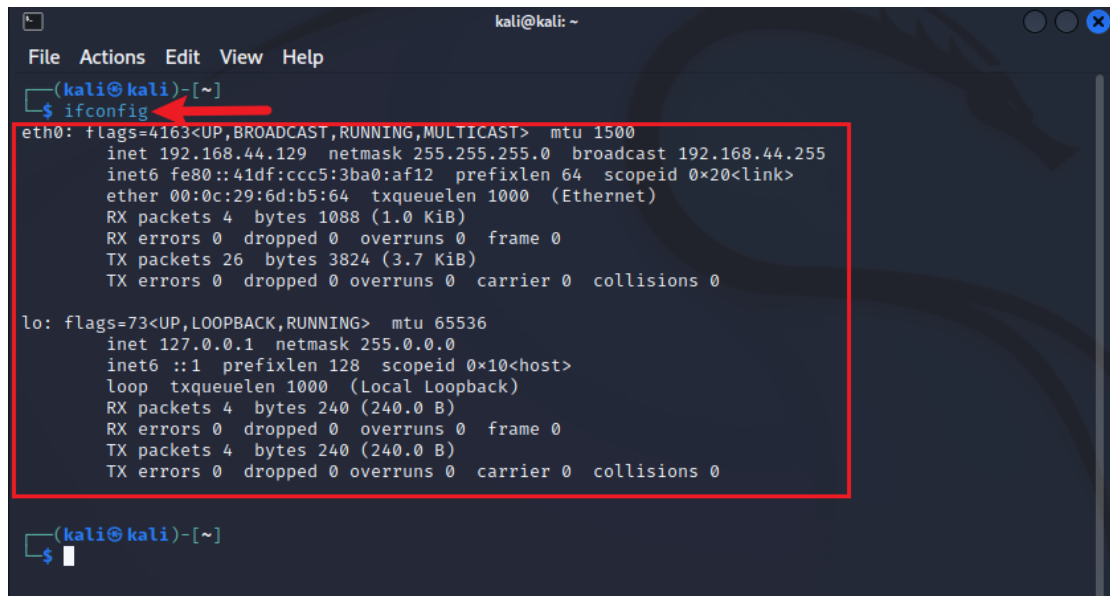


2、查看两虚拟机的地址信息

(1) 在 kali 的命令行输入:

```
ifconfig
```

以查看 kali 的地址信息。

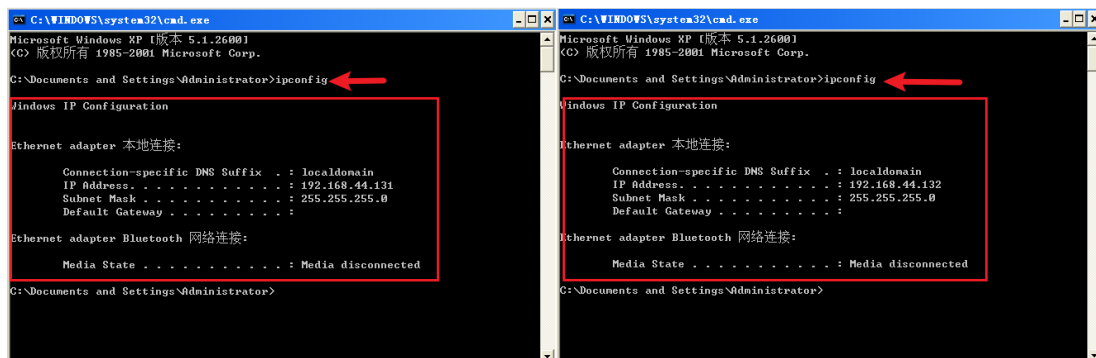


```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.44.129 netmask 255.255.255.0 broadcast 192.168.44.255  
    inet6 fe80::41df:ccc5:3ba0:af12 prefixlen 64 scopeid 0<link>  
    ether 00:0c:29:6d:b5:64 txqueuelen 1000 (Ethernet)  
    RX packets 4 bytes 1088 (1.0 KiB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 26 bytes 3824 (3.7 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 4 bytes 240 (240.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 4 bytes 240 (240.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
(kali@kali)-[~]  
$
```

(2) 在 WinXP1、WinXP2 的命令行输入:

```
ipconfig
```

以查看 WinXP1、WinXP2 的地址信息。



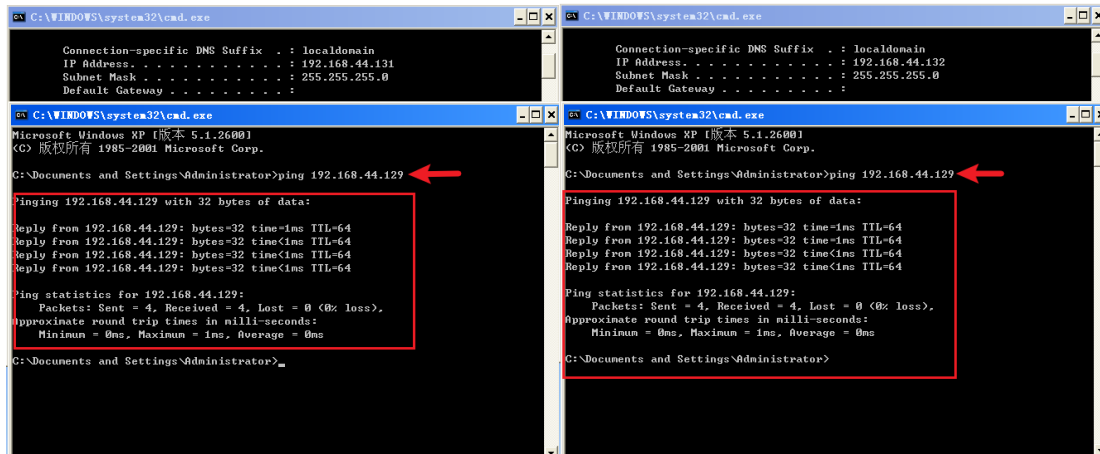
```
C:\WINDOWS\system32\cmd.exe  
Microsoft Windows XP [版本 5.1.2600]  
(C) 版权所有 1985-2001 Microsoft Corp.  
C:\Documents and Settings\Administrator>ipconfig  
  
Windows IP Configuration  
  
Ethernet adapter 本地连接:  
    Connection-specific DNS Suffix . : localdomain  
    IP Address . . . . . : 192.168.44.131  
    Subnet Mask . . . . . : 255.255.255.0  
    Default Gateway . . . . . :  
  
Ethernet adapter Bluetooth 网络连接:  
    Media State . . . . . : Media disconnected  
C:\Documents and Settings\Administrator>  
  
C:\WINDOWS\system32\cmd.exe  
Microsoft Windows XP [版本 5.1.2600]  
(C) 版权所有 1985-2001 Microsoft Corp.  
C:\Documents and Settings\Administrator>ipconfig  
  
Windows IP Configuration  
  
Ethernet adapter 本地连接:  
    Connection-specific DNS Suffix . : localdomain  
    IP Address . . . . . : 192.168.44.132  
    Subnet Mask . . . . . : 255.255.255.0  
    Default Gateway . . . . . :  
  
Ethernet adapter Bluetooth 网络连接:  
    Media State . . . . . : Media disconnected  
C:\Documents and Settings\Administrator>
```

3、两台 WinXP ping Kali

在 WinXP1、WinXP2 的命令行输入:

```
ping 192.168.44.129
```

以 ping 通 Kali。



4、禁止所有主机 ping 本地主机

(1) 在 kali 的命令行输入：

```
sudo iptables -A INPUT -p icmp -j DROP
```

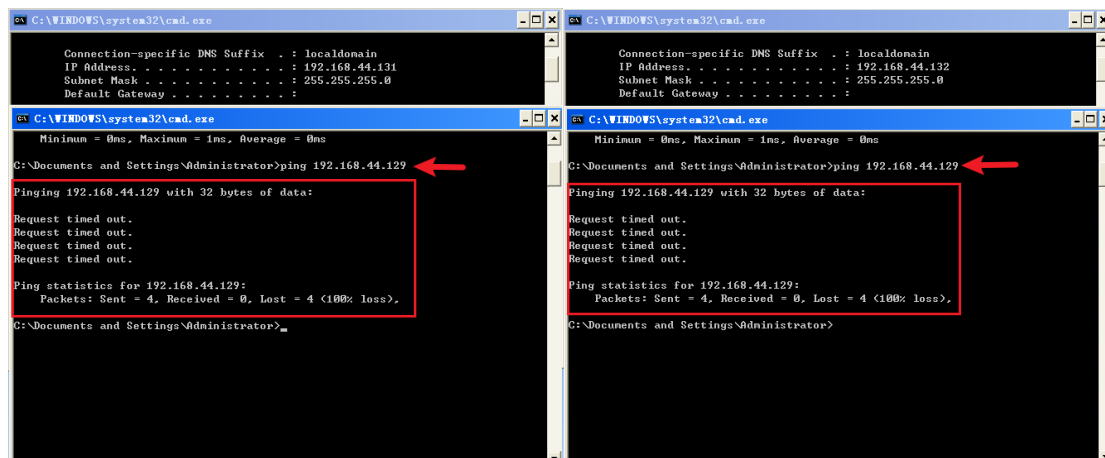
以实现禁止所有主机 ping 通本机。



(2) 在 WinXP1、WinXP2 的命令行输入：

```
ping 192.168.44.129
```

可以看到已经不能 ping 通。



5、仅允许某特定 IP 主机 ping 本地主机

(1) 在 kali 的命令行输入：

```
sudo iptables -I INPUT -p icmp -s 192.168.44.131 -j ACCEPT
```

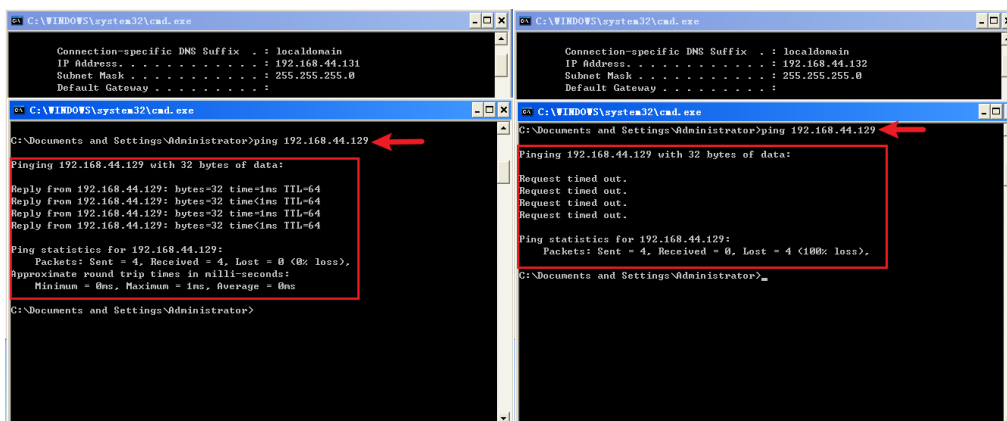
以实现仅允许 IP 地址为 192.168.44.131 的主机 ping 通。

```
(kali@kali)-[~]  
$ sudo iptables -I INPUT -p icmp -s 192.168.44.131 -j ACCEPT
```

(2) 在 WinXP1、WinXP2 的命令行输入:

```
ping 192.168.44.129
```

WinXP (192.168.44.131) 可以 ping 通 Kali, WinXP (192.168.44.132) 不能 ping 通 Kali。



6、允许每 10 秒钟通过 1 个 ping 包

(1) 在 kali 的命令行输入:

```
sudo iptables -F
```

以清除之前的规则。

```
(kali@kali)-[~]  
$ sudo iptables -F  
[sudo] password for kali:
```

(2) 在 kali 的命令行输入:

```
sudo iptables -I INPUT -p icmp -m limit --limit 6/min --limit-burst 1 -j ACCEPT
```

```
sudo iptables -A INPUT -p icmp -j DROP
```

以实现允许每 10s 通过一个 ping 包。

```
(kali@kali)-[~]
$ sudo iptables -I INPUT -p icmp -m limit --limit 6/min --limit-burst 1 -j ACCEPT
(kali@kali)-[~]
$ sudo iptables -A INPUT -p icmp -j DROP
```

(3) 在 WinXP1 (192.168.44.131) 编写 ping.vbs 文件放到 C 盘根目录，双击运行。

```
Set shell = WScript.CreateObject("WScript.Shell")
'脚本解析引擎，创建一个命令窗口对象
Dim ipstr,pingstr
ipstr="192.168.44.129"
'需要 ping 的目录 Ip
Set fso=createobject("scripting.filesystemobject")
'创建了一个 FSO 对象，然后中用它来读写文本文件，删除文件等
Set file = fso.createtextfile("c:\ping" & ipstr & ".txt")
'用 fso 创建一个文件
Set myping=shell.Exec("ping -t " & ipstr )
'执行的命令
while Not myping.StdOut.AtEndOfStream
'如果文件指针位于 命令行文件末，则返回 True；否则返回 False
    strLine = myping.StdOut.ReadLine()
    '读取此行
    file.writeline date & " "& time & chr(9) & strLine
    '写入文件
Wend
```

(4) 运行一段时间后，在 WinXP1 (192.168.44.131) 的 C 盘根目录下查看 ping192.168.44.129.txt 文档。第一个包 23: 52: 59，最后一个包 23: 54: 10，7 个包 71s，即是 10s 通过一个 ping 包。

```
ping192.168.44.129.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
2024-11-13 23:52:59
2024-11-13 23:52:59 Pinging 192.168.44.129 with 32 bytes of data:
2024-11-13 23:52:59 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:53:04 Request timed out.
2024-11-13 23:53:09 Request timed out.
2024-11-13 23:53:10 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:53:16 Request timed out.
2024-11-13 23:53:21 Request timed out.
2024-11-13 23:53:22 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:53:28 Request timed out.
2024-11-13 23:53:33 Request timed out.
2024-11-13 23:53:34 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:53:40 Request timed out.
2024-11-13 23:53:45 Request timed out.
2024-11-13 23:53:46 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:53:52 Request timed out.
2024-11-13 23:53:57 Request timed out.
2024-11-13 23:53:58 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:54:04 Request timed out.
2024-11-13 23:54:09 Request timed out.
2024-11-13 23:54:10 Reply from 192.168.44.129: bytes=32 time=1ms TTL=64
2024-11-13 23:54:19 Request timed out.
2024-11-13 23:54:19 Ping statistics for 192.168.44.129:
2024-11-13 23:54:19 Packets: Sent = 20, Received = 7, Lost = 13 (65% loss),
2024-11-13 23:54:19 Approximate round trip times in milli-seconds:
2024-11-13 23:54:19 Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

7、阻断来自某个 MAC 地址的数据包

(1) 在 kali 的命令行输入：

```
sudo iptables -F
```

以清除之前的规则。

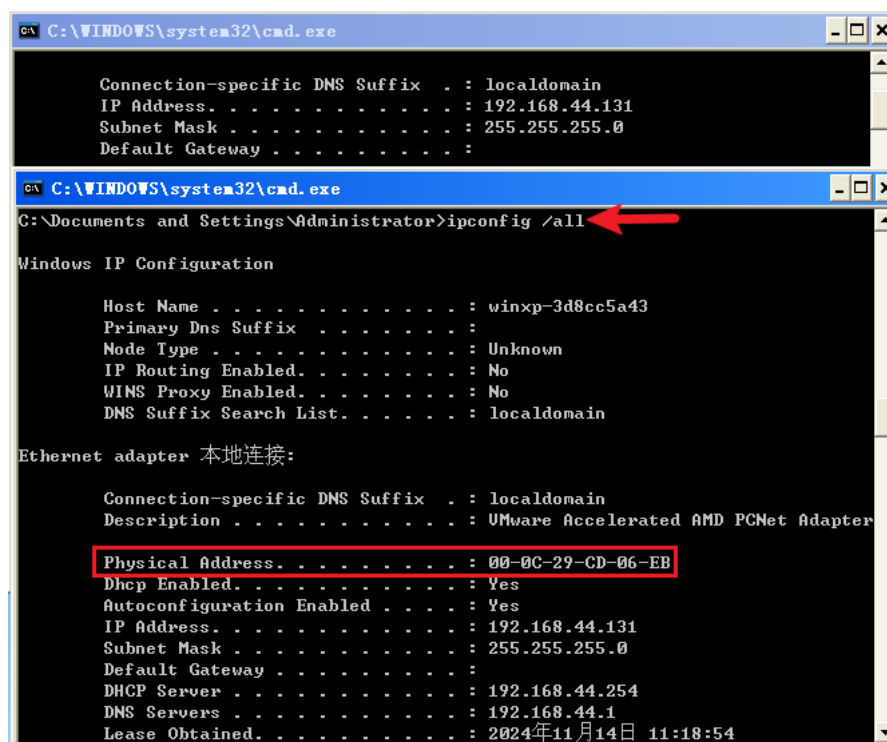


```
(kali@kali)-[~]  
$ sudo iptables -F  
[sudo] password for kali:
```

(2) 在 WinXP1 (192.168.44.131) 命令行输入：

```
ipconfig /all
```

以查看 WinXP1 (192.168.44.131) 的 MAC 地址为 00:0C:29:CD:06:EB。

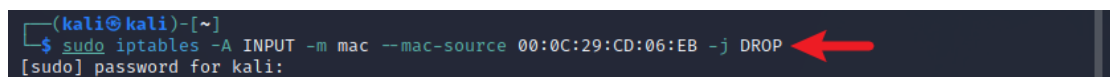


```
C:\WINDOWS\system32\cmd.exe  
Connection-specific DNS Suffix . : localdomain  
IP Address . . . . . : 192.168.44.131  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . :  
  
C:\WINDOWS\system32\cmd.exe  
C:\Documents and Settings\Administrator>ipconfig /all  
Windows IP Configuration  
  
Host Name . . . . . : winxp-3d8cc5a43  
Primary Dns Suffix . . . . . :  
Node Type . . . . . : Unknown  
IP Routing Enabled. . . . . : No  
WINS Proxy Enabled. . . . . : No  
DNS Suffix Search List. . . . . : localdomain  
  
Ethernet adapter 本地连接:  
  
Connection-specific DNS Suffix . : localdomain  
Description . . . . . : VMware Accelerated AMD PCNet Adapter  
Physical Address. . . . . : 00-0C-29-CD-06-EB  
Dhcp Enabled. . . . . : Yes  
Autoconfiguration Enabled . . . . : Yes  
IP Address. . . . . : 192.168.44.131  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . :  
DHCP Server . . . . . : 192.168.44.254  
DNS Servers . . . . . : 192.168.44.1  
Lease Obtained. . . . . : 2024年11月14日 11:18:54
```

(3) 在 kali 的命令行输入：

```
sudo iptables -A INPUT -m mac --mac-source 00:0C:29:CD:06:EB -j DROP
```

以实现阻断来自 MAC 地址为 00:0C:29:CD:06:EB 的主机的数据包的目的。

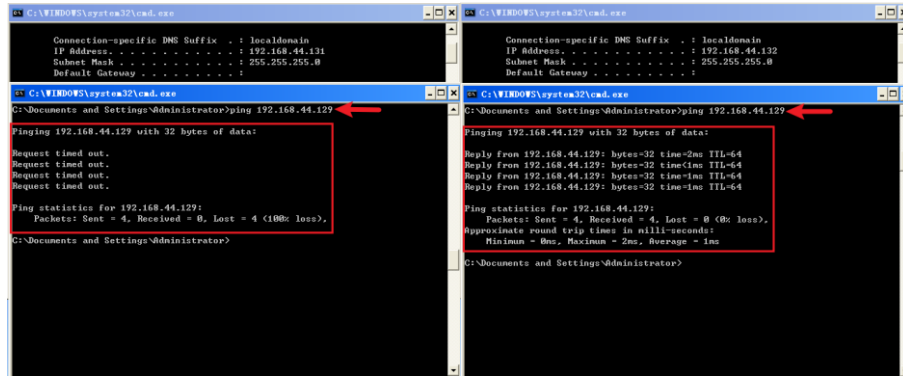


```
(kali@kali)-[~]  
$ sudo iptables -A INPUT -m mac --mac-source 00:0C:29:CD:06:EB -j DROP  
[sudo] password for kali:
```

(4) 在 WinXP1、WinXP2 命令行输入：

```
ping 192.168.44.129
```

可以看到 WinXP1 (192.168.44.131, 00:0C:29:CD:06:EB) 已经不能 ping 通, WinXP2 可以正常 ping 通。



8、配置 iptables，允许特定的远端客户机器 SSH 连接本地主机

(1) 在 kali 的命令行输入：

```
sudo iptables -F
```

以清除之前的规则。



(2) 在 kali 的命令行输入：

```
sudo /etc/init.d/ssh start
```

以启动 ssh 服务，使得客户端可以通过 22 端口远程连接。



(3) 在 kali 的命令行输入：

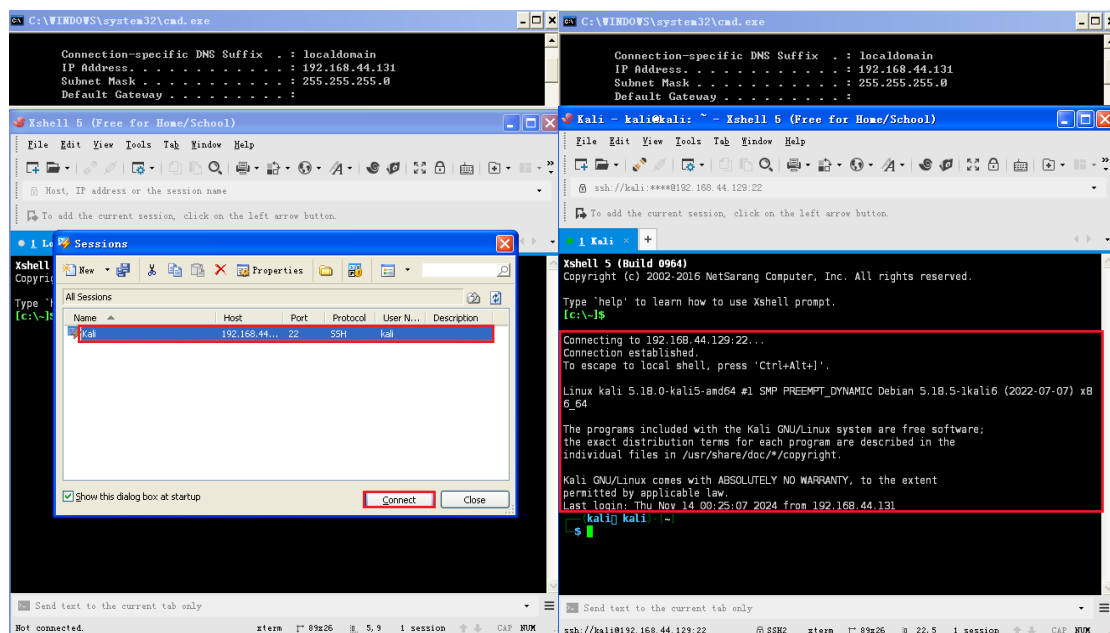
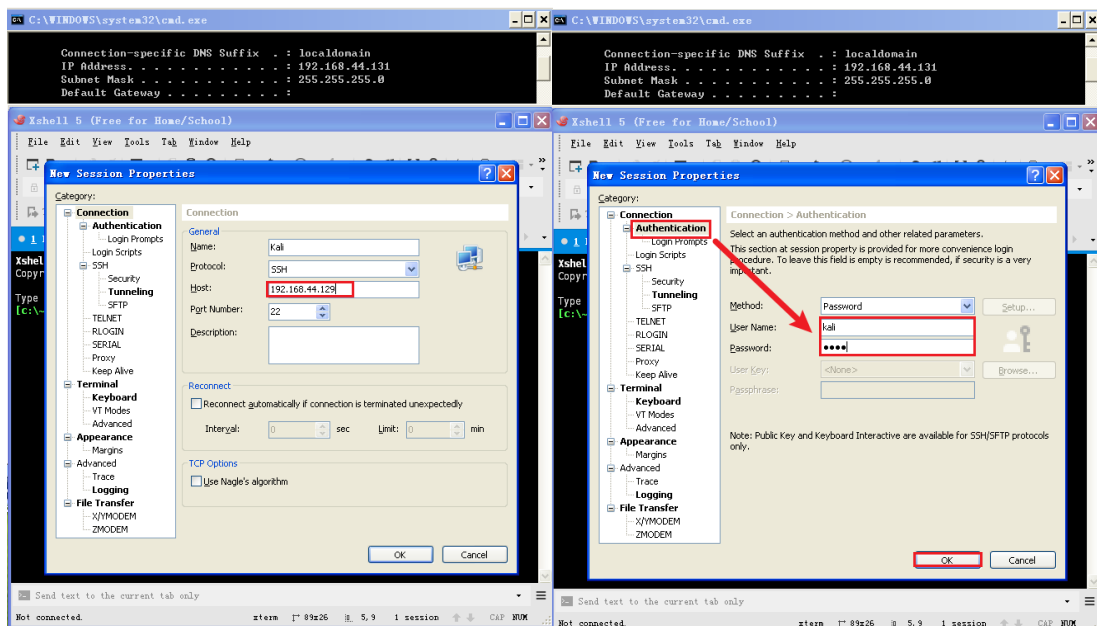
```
sudo /etc/init.d/ssh status
```

以查看 ssh 服务器状态。

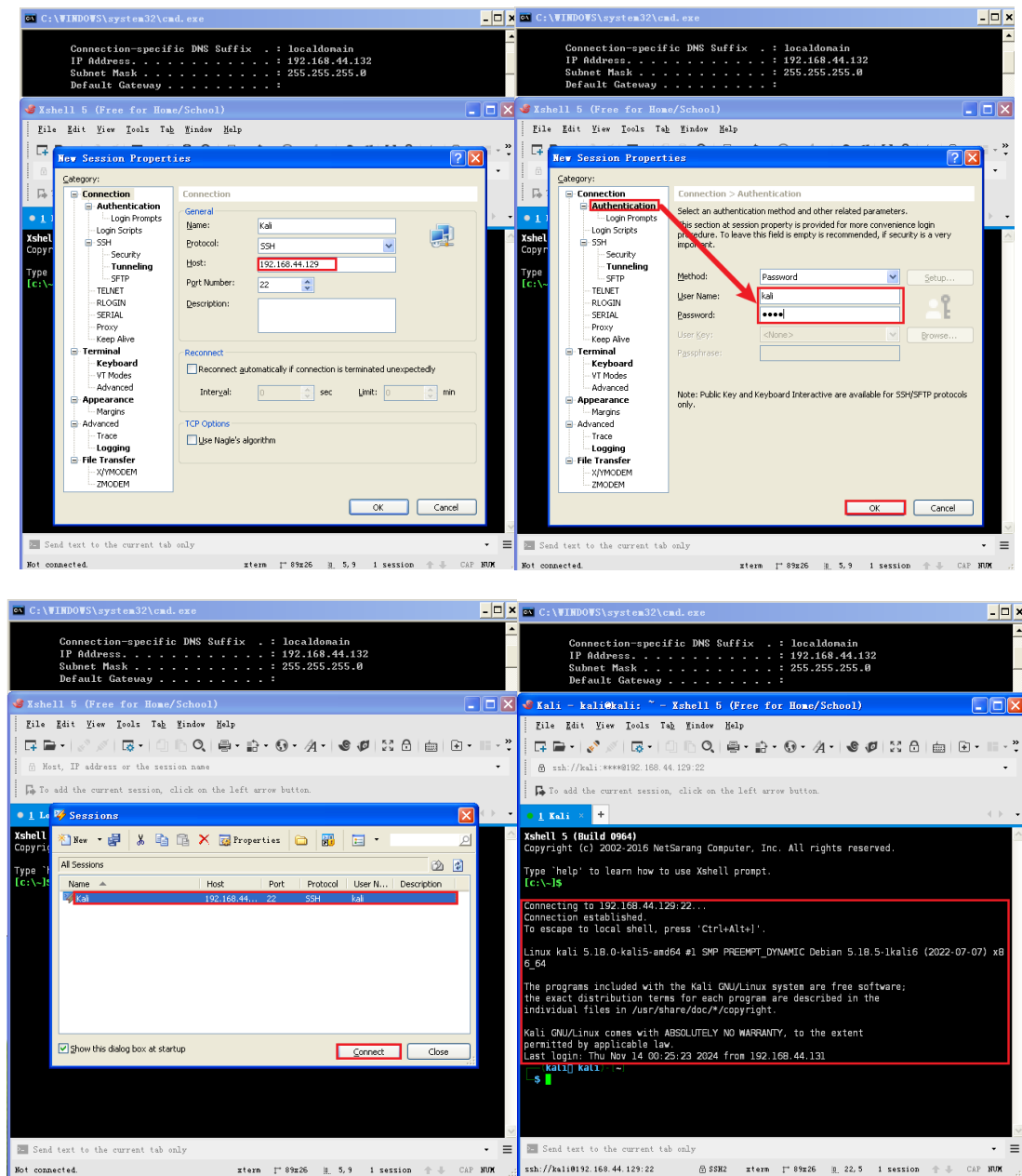
```
(kali@kali)~$ sudo /etc/init.d/ssh status
ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/lib/systemd/system/ssh.service; disabled; vendor preset: disabled)
Active: active (running) since Thu 2024-11-14 00:09:04 EST; 38s ago
Docs: man:sshd(8)
      man:sshd_config(5)
Process: 39973 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
Main PID: 39986 (sshd)
Tasks: 1 (limit: 2283)
Memory: 1.9M
CPU: 27ms
CGroup: /system.slice/ssh.service
        └─39986 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 14 00:09:04 kali systemd[1]: Starting OpenBSD Secure Shell server...
Nov 14 00:09:04 kali sshd[39986]: Server listening on 0.0.0.0 port 22.
Nov 14 00:09:04 kali sshd[39986]: Server listening on :: port 22.
Nov 14 00:09:04 kali systemd[1]: Started OpenBSD Secure Shell server.
```

(4) 在 WinXP1 (192.168.44.131) 中用 Xshell 进行 ssh 远程连接 Kali, 连接成功。



(5) 在 WinXP2 (192.168.44.132) 中用 Xshell 进行 ssh 远程连接 Kali, 连接成功。



(6) 在 kali 的命令行输入:

```
sudo /etc/init.d/ssh status
```

以查看 ssh 服务器状态。客户端 1、2 分别连接后, SSH 服务器上会留下记录。可以看到 WinXP1 (192.168.44.131) 和 WinXP2 (192.168.44.132) 均连接成功。

```
(kali@kali)-[~]
$ sudo /etc/init.d/ssh status
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; disabled; vendor preset: disabled)
   Active: active (running) since Thu 2024-11-14 00:09:04 EST; 24min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 39973 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 39986 (sshd)
     Tasks: 1 (limit: 2283)
    Memory: 5.8M
       CPU: 271ms
   CGroup: /system.slice/ssh.service
           └─39986 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 14 00:09:04 kali systemd[1]: Starting OpenBSD Secure Shell server...
Nov 14 00:09:04 kali sshd[39986]: Server listening on 0.0.0.0 port 22.
Nov 14 00:09:04 kali sshd[39986]: Server listening on :: port 22.
Nov 14 00:09:04 kali systemd[1]: Started OpenBSD Secure Shell server.
Nov 14 00:25:07 kali sshd[43911]: Accepted password for kali from 192.168.44.131 port 1085 ssh2
Nov 14 00:25:07 kali sshd[43911]: pam_unix(sshd:session): session opened for user kali(uid=10...uid=0)
Nov 14 00:25:23 kali sshd[44022]: Accepted password for kali from 192.168.44.131 port 1086 ssh2
Nov 14 00:25:23 kali sshd[44022]: pam_unix(sshd:session): session opened for user kali(uid=10...uid=0)
Nov 14 00:31:28 kali sshd[45469]: Accepted password for kali from 192.168.44.132 port 1085 ssh2
Nov 14 00:31:28 kali sshd[45469]: pam_unix(sshd:session): session opened for user kali(uid=10...uid=0)
Hint: Some lines were ellipsized, use -l to show in full.
```

(7) 在 kali 的命令行输入:

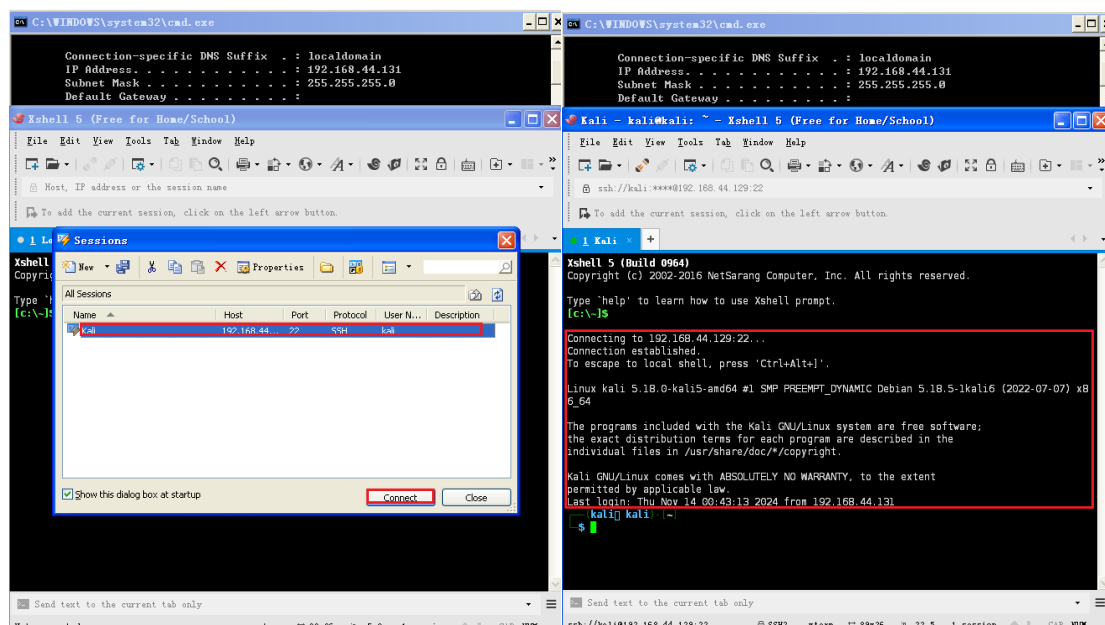
```
sudo iptables -I INPUT -p tcp --dport 22 -s 192.168.44.131 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

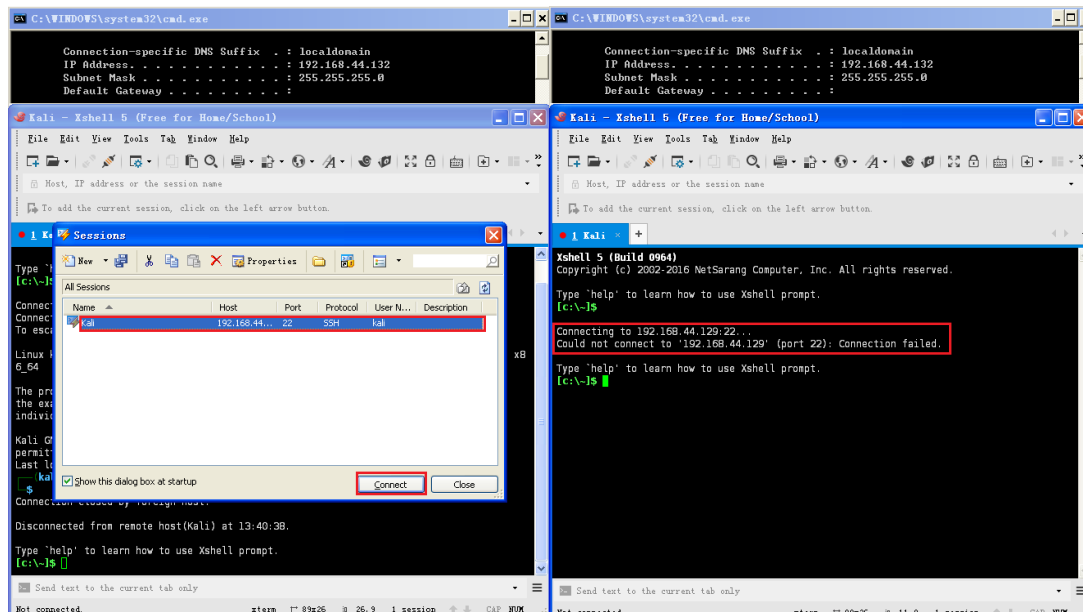
以实现只允许 WinXP1 (192.168.44.131) 进行 ssh 连接的目的

```
(kali@kali)-[~]
$ sudo iptables -I INPUT -p tcp --dport 22 -s 192.168.44.131 -j ACCEPT
(kali@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

(8) 在 WinXP1 (192.168.44.131) 中用 Xshell 进行 ssh 远程连接 Kali, 连接成功。



(9) 在 WinXP2 (192.168.44.132) 中用 Xshell 进行 ssh 远程连接 Kali, 连接失败。



二、基于 iptables 配置 NAT，实现内网服务的发布实验

(一) 实验内容

- 1、配置网络环境，测试网络连通情况。
- 2、开启防火墙路由转发，设置 iptables 规则。
- 3、验证公网主机能否通过访问防火墙获得内网服务。

(二) 实验环境

三台虚拟机：1) 防火墙部署在 kali linux 2022.3 (192.168.44.129)；2) 安装 nginx 的 WinXP1 (192.168.44.131) 作为内网服务器；3) WinXP2 (192.168.44.132) 公网主机

防火墙：linux 内核防火墙的命令行工具 iptables

Windows 平台 web 服务器：nginx

(三) 实验场景

假设 PC2 不能直接访问局域网内的电脑 PC1 上的服务，但 PC2 可以与 firewall 上的公网 IP: 192.168.2.254 相互通讯，由此可以进行 NAT 的设置，让 PC2 访问 firewall 的公网地址上的服务，再由 firewall 对数据进行处理，将访问请求发送到内网 PC1 上。



(四) 实验步骤

1、配置网络环境

(1) 给作为 firewall 的 Kali 增设网卡，并设置两张网卡处在不同的网络中。网卡 1 使用“仅主机模式”，网卡 2 使用“NAT”。



(2) 在 firewall (kali) 的命令行输入：

```
ifconfig
```

以查看 IP。可知，网关的配置：内网 IP: 192.168.56.128 子网掩码: 255.255.255.0；公网 IP: 192.168.80.156 子网掩码: 255.255.255.0。

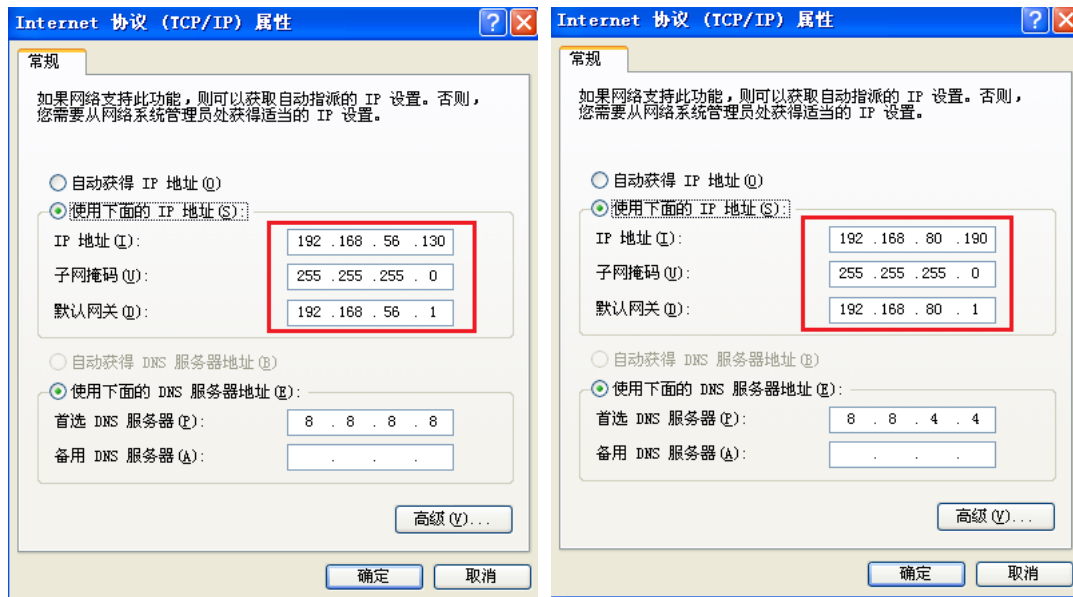
```

(kali@kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.128 netmask 255.255.255.0 broadcast 192.168.56.255
    inet6 fe80::20c:29ff:fe6d:b564 prefixlen 64 scopeid 0<link>
    ether 00:0c:29:6d:b5:64 txqueuelen 1000 (Ethernet)
    RX packets 513 bytes 63257 (61.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 36 bytes 3928 (3.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.80.156 netmask 255.255.255.0 broadcast 192.168.80.255
    inet6 fe80::f497:9b02:38bd:84d5 prefixlen 64 scopeid 0<link>
    ether 00:0c:29:6d:b5:6e txqueuelen 1000 (Ethernet)
    RX packets 86 bytes 11890 (11.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 104 bytes 9986 (9.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 16 bytes 1296 (1.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 16 bytes 1296 (1.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
  
```

(3) 配置 WinXP1、WinXP2 的网络连接。



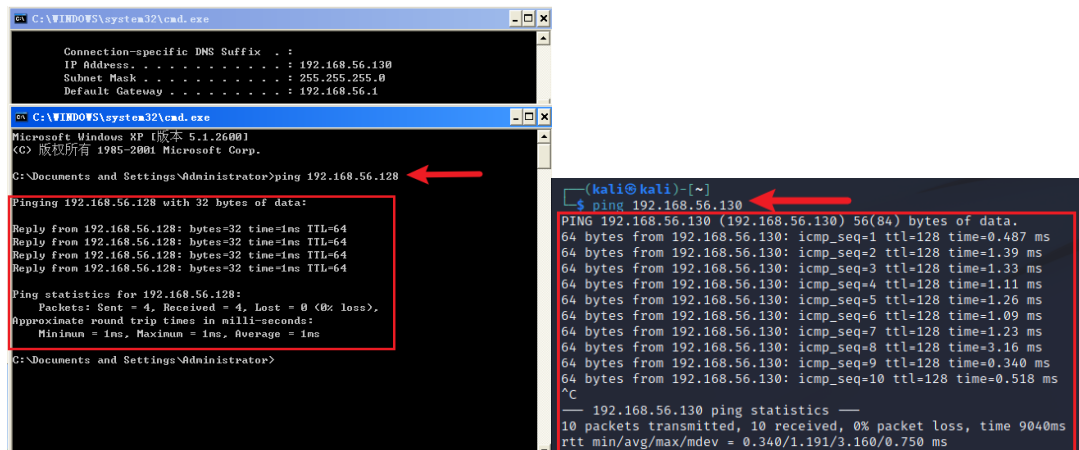
(4) 在 WinXP1 命令行输入：

```
ping 192.168.56.128
```

在 firewall (kali) 命令行输入：

```
ping 192.168.56.130
```

WinXP1 可以与 firewall 互相 ping 通。



(5) 在 WinXP2 命令行输入：

```
ping 192.168.80.156
```

在 firewall (kali) 命令行输入：

```
ping 192.168.80.190
```

WinXP2 可以与 firewall 互相 ping 通。

The image shows two terminal windows side-by-side. The left window is a Windows XP command prompt (cmd.exe) showing the IP configuration for 192.168.80.190 and the output of a ping command to 192.168.80.156. The output shows four successful replies with 32 bytes of data, a TTL of 64, and a round trip time of approximately 1ms. The right window is a Kali Linux terminal (kali@kali) showing the output of a ping command to 192.168.80.190. The output shows 12 successful pings with 56(84) bytes of data, a TTL of 128, and a round trip time of approximately 1ms. Both windows have red arrows pointing to the ping command and its output.

(6) 在 WinXP1 命令行输入:

```
ping 192.168.80.190
```

在 WinXP2 命令行输入:

```
ping 192.168.56.130
```

WinXP1 与 WinXP2 无法互相 ping 通。

The image shows two terminal windows side-by-side. The left window is a Windows XP command prompt (cmd.exe) showing the IP configuration for 192.168.56.130 and the output of a ping command to 192.168.80.190. The output shows four 'Request timed out' messages and a round trip time of 1ms. The right window is a Windows XP command prompt (cmd.exe) showing the IP configuration for 192.168.80.190 and the output of a ping command to 192.168.56.130. The output shows four 'Request timed out' messages and a round trip time of 1ms. Both windows have red arrows pointing to the ping command and its output.

2、开启 firewall 的路由转发

在 firewall (kali) 的命令行输入:

```
su root
```

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

以开启防火墙的路由转发。

```
(kali㉿kali)-[~]  
$ su root  
Password:  
(root㉿kali)-[/home/kali]  
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

3、设置 NAT 规则

(1) 在 firewall (kali) 的命令行输入：

```
iptables -t nat -A PREROUTING -i ens38 -p tcp -d 192.168.80.156 --dport 80 -j DNAT --to-destination 192.168.56.128
```

以实现对公网来的访问数据设置 DNAT 规则，将其访问目标地址通过 NAT 修改为提供服务的 IP，以便将请求转发到内网对应 IP 上。

```
(root㉿kali)-[/home/kali]  
# iptables -t nat -A PREROUTING -i ens38 -p tcp -d 192.168.80.156 --dport 80 -j DNAT --to-destination 192.168.56.128
```

(2) 在 firewall (kali) 的命令行输入：

```
iptables -t nat -A POSTROUTING -o ens38 -p tcp -j SNAT --to-source 192.168.80.156
```

以设置一条 SNAT 的规则，以便内网对公网数据访问的回应信息能够到达请求的客户。

```
(root㉿kali)-[/home/kali]  
# iptables -t nat -A POSTROUTING -o ens38 -p tcp -j SNAT --to-source 192.168.80.156
```

(3) 在 firewall (kali) 的命令行输入：

```
iptables -A FORWARD -i ens38 -p tcp -d 192.168.56.128 --dport 80 -j ACCEPT
```

```
iptables -A FORWARD -p tcp -m state --state established,related -j ACCEPT
```

以在 FORWARD 链上设置允许访问内网的规则。

```
(root㉿kali)-[/home/kali]  
# iptables -A FORWARD -i ens38 -p tcp -d 192.168.56.128 --dport 80 -j ACCEPT  
(root㉿kali)-[/home/kali]  
# iptables -A FORWARD -p tcp -m state --state established,related -j ACCEPT
```

4、启动 WinXP1 的 nginx 开启 http 服务

在 WinXP1 命令行输入：


```
start nginx
```

```
netstat -an
```

```
C:\WINDOWS\system32\cmd.exe

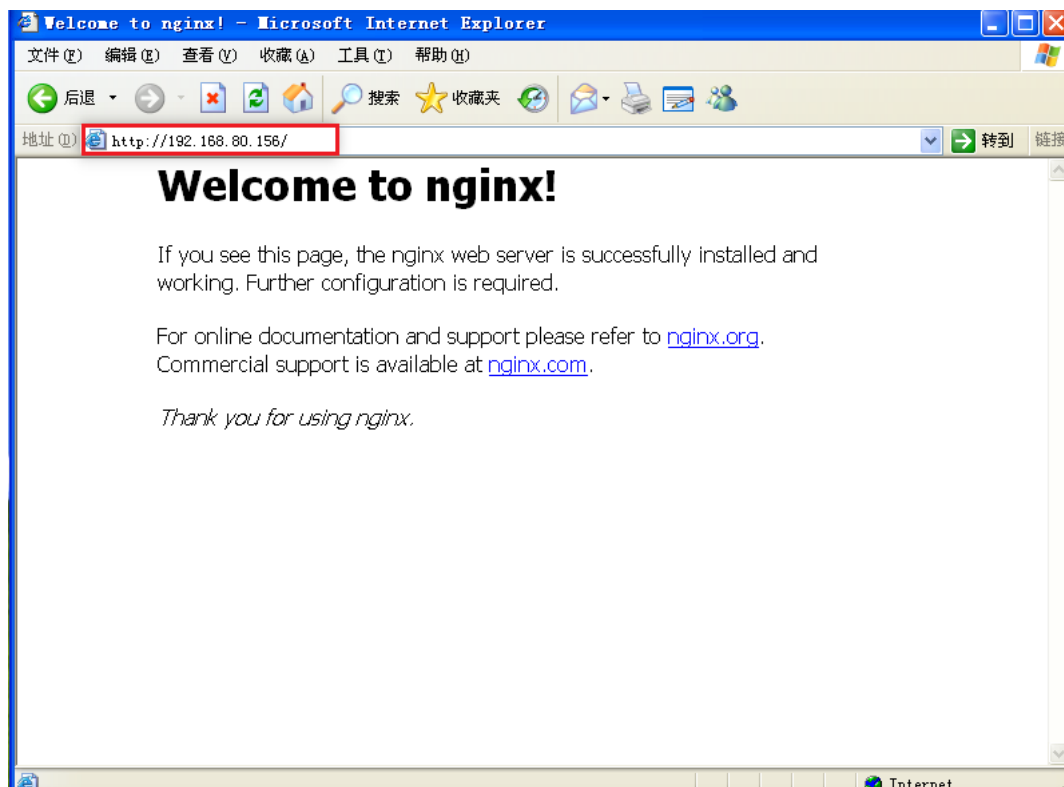
Connection-specific DNS Suffix . : 
IP Address. . . . . : 192.168.56.130
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.56.1

C:\Documents and Settings\Administrator\桌面\nginx-1.26.2\nginx-1.26.2>start nginx
C:\Documents and Settings\Administrator\桌面\nginx-1.26.2\nginx-1.26.2>netstat -an

Active Connections

Proto Local Address Foreign Address State
TCP 0.0.0.0:80 0.0.0.0:0 LISTENING
TCP 0.0.0.0:135 0.0.0.0:0 LISTENING
TCP 0.0.0.0:445 0.0.0.0:0 LISTENING
TCP 127.0.0.1:1029 0.0.0.0:0 LISTENING
TCP 192.168.56.130:139 0.0.0.0:0 LISTENING
UDP 0.0.0.0:445 *:*
UDP 0.0.0.0:5000 *:*
UDP 0.0.0.0:1025 *:*
UDP 0.0.0.0:1050 *:*
UDP 0.0.0.0:4500 *:*
UDP 127.0.0.1:123 *:*
UDP 127.0.0.1:1026 *:*
UDP 127.0.0.1:1048 *:*
UDP 127.0.0.1:1900 *:*
UDP 192.168.56.130:123 *:*
```

5、在 WinXP2 浏览器输入 192.168.80.156，即 firewall 的公网 IP。可以看到启动 WinXP1 的 http 服务后，WinXP2 可以通过使用防火墙公网 IP 访问内网服务。



6、解释上述使用的 iptables 规则，并分析 NAT 的实现原理

(1)

```
iptables -t nat -A PREROUTING -i ens38 -p tcp -d 192.168.80.156 --dport 80 -j DNAT --to-destination 192.168.56.128
```

这个 iptables 规则的作用是将进入 ens38 接口，目标地址为 192.168.80.156，目标端口为 80 的 TCP 数据包的目标地址修改为 192.168.56.128，用于实现端口转发。

(2)

```
iptables -t nat -A POSTROUTING -o ens38 -p tcp -j SNAT --to-source 192.168.80.156
```

这个 iptables 规则的作用是将离开 ens38 接口的所有 TCP 数据包的源地址修改为 192.168.80.156。在 NAT 环境中，将内部主机的私有 IP 地址替换为网关的公共 IP 址，以便在 Internet 上进行通信。

(3)

```
iptables -A FORWARD -i ens38 -p tcp -d 192.168.56.128 -dport 80 -j ACCEPT
```

```
iptables -A FORWARD -p tcp -m state --state established,related -j ACCEPT
```

这两个 iptables 规则用于配置包转发策略。第一个规则允许进入接口 ens38 的目标地址为 192.168.56.128，目标端口为 80 的 TCP 数据包通过系统。第二个规则允许已建立的连接或与已建立连接相关的 TCP 数据包通过系统的转发。这通常用于处理与先前已建立的连接相关的数据包，以确保它们被正确转发。

因此可知，NAT 的实现就是通过防火墙阻隔内网和外网之间的通讯，并为内网的所有 IP 地址分配相对应的外网网段地址。防火墙起到了地址转换的作用，当其对外网进行收发信息的时候会将内网地址转化为外网分配 IP，或将外网 IP 根据对应表转化为内网地址。从而实现信息中转功能。

四、总结及问题

(一) 总结

通过本次实验，学习到了 iptables 的相关用法和 NAT 的原理知识。

iptables 是 Linux 操作系统中用于配置 IPv4 包过滤规则、进行网络地址转换 (NAT) 和包队列处理的强大工具。通过 iptables 命令，用户可以定义防火墙规则，控制数据包的流动，并实现网络安全策略，包括端口转发、地址转换等功能，以保护系统免受网络攻击。iptables 的灵活性和可定制性使其成为 Linux 系统中网络管理和安全配置的关键组件。

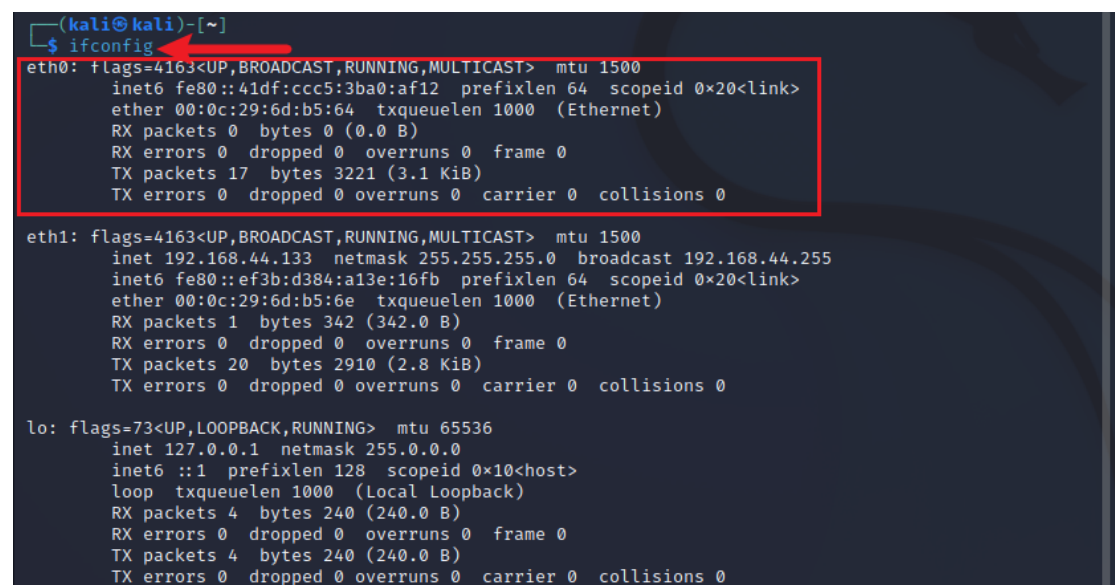
NAT 将内部私有网络中的局域网 IP 地址映射为外部公共网络上的一个或多个公共 IP 地址。这通过在数据包经过设备时修改其源或目标 IP 地址来实现。NAT 允许多个内部设备共享同一个公共 IP 地址，有效地解决了 IPv4 地址短缺的问题。它的基本原理是在数据包经过设备时，根据预定义的转换规则，更新数据包的 IP 地址信息，使其在内外网络之间传递时能够正确路由。这种技术广泛用于家庭网络和企业环境，为多个设备提供对互联网的访问，同时提供一定程度的网络安全和隐私保护。

(二) 问题及解决

1、问题

(1) putty 无法正常安装。

(2) 在给作为 firewall 的 kali 增设网卡后，输入 ifconfig 查看 IP 时，eth0 无 IPV4 地址。



```
(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::41df:ccc5:3ba0:af12 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:6d:b5:64 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 17 bytes 3221 (3.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.44.133 netmask 255.255.255.0 broadcast 192.168.44.255
    inet6 fe80::ef3b:d384:a13e:16fb prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:6d:b5:6e txqueuelen 1000 (Ethernet)
    RX packets 1 bytes 342 (342.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 20 bytes 2910 (2.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 4 bytes 240 (240.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4 bytes 240 (240.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

2、解决

- (1) 安装 Xshell5。
- (2) 重新编辑虚拟网络编辑器。